成都站 赛前小技巧与题型补丁 2026

先从题意找模型，再查代码。优先直接采用我原来验证使用过的实现，只修具体问题；基础树存储和容器用法与算法同样重要。

这是短补充资料，配合第一轮 24 页急用版使用；未在本册展开的源码按编号到仓库查阅。

原版 inline ll read()保留；按题目 n/T 读取常规整数，其他输入用 cin。关闭 iostream 同步后，不混用 cin 与 getchar。

纸质资料可携带数量、规格：待赛站确认。生成文件不表示全部允许带入比赛。

先按操作选容器

题目需要	首先考虑
每次取当前最小 / 最大	小根堆 / 大根堆
动态有序去重 / 允许重复	set / multiset
动态频率计数	map / unordered_map
查前驱、后继、第一个不小于 x	set / multiset
先进先出 / 后进先出	queue / stack
两端增删 / 固定窗口候选	deque / 单调队列
字符串前缀 / 最大异或数对	字符串 Trie / 二进制 Trie
单点加、前缀与区间和	Fenwick
静态区间 min/max/gcd	ST 表
动态区间合并信息 / 整段修改	线段树 / Lazy 线段树
连通关系 / 无权最短路	DSU / BFS
0/1 权 / 非负一般权最短路	0-1 BFS / Dijkstra

复杂度先看 n、q、值域和状态数；数量级只是筛选，不能替代证明或实测。

从题意到实现的六步

1 看约束

很小先穷举；n≈40 考虑折半；集合小考虑状压；大 n 寻找线性或 nlogn；值域小计数、大值域离散化。

2 看操作

连续段→前缀/差分/数据结构；最少步→下界构造/BFS/DP；可达→不变量+充分性；轮流行动→先核对终止与胜负规则。

3 写暴力

枚举合法对象，找重复计算；问未来究竟还需要哪些历史信息。

4 补证明

构造要合法且达到目标；贪心写交换；二分证单调；DP 证状态足够并划分互斥完整。

5 选源码

先用已有熟悉代码。写清 1/0 下标、闭/半开区间、初始化、值域、输出含义。

6 找反例

最小规模、全相等、全负、边界、断图/重边、多测；小数据暴力与优化对拍，记录最小反例。

DP 状态检查：保存什么 / 漏了什么 / 初值和不可达 / 转移是否重计 / 遍历依赖 / 答案位置 / max、min 还是 sum / 是否取模 / 能否滚动。

WA 先核对语义与边界；RE 查下标、空容器和递归深度；TLE 先重算复杂度；多测清实际使用过的状态。

每个代码模块单独使用。同名全局 a/g/tree/init 等来自不同模板，拼到同一题前只保留所需实现或按题目改名。本地 #include 在打印中已递归展开；无需照抄文件依赖。

我的代码习惯与 DP 状态识别

原代码验证过就直接采用：ll、1 下标、全局 a/g/dp/cnt、独立函数；原 Fenwick 的 struct/vector 同样保留。树用 vector<ll> g[N]存双向边；新题先 init 清上一组状态。

线段树沿用 pushup、pushdown、build、modify、query；DSU 沿用 init、find、unite、same。已验证的原实现与新增修复均有来源记录，不为统一外观改写。

识别信号	必须保存的状态 / 编号
一段只能接前一段	以 i 结尾的最优值 DPC01
只能右下走	到(i,j)的方案数/代价 DPC02
每件最多一次 / 无限 / 有限 / 分组	容量与阶段，遍历方向不同 DPC03–06
最后一次合并把区间切开	区间[l,r]与切分点 k DPC07
父子选择互斥	子树根选/不选 DPC08
依赖无环	拓扑序上的最优前驱 DPC09
小集合与当前位置	mask 和 last DPC10
区间很大但数位少	pos/tight/started 和题目状态 DPC11
保留原序 / 两序列匹配	末尾值或两个前缀长度 DPC12
历史只影响当前阶段	位置与有限状态 DPC13
问方案数 / 随机收益	互斥计数 / 概率加权与自环移项 DPC14–15

状态相同必须拥有相同的后续选择。max/min 保留最优；计数相加但防重；不可达要单独表示。详推导与完整代码见 V2 相同编号。

题型识别与代码索引

稳定编号与内容	页
MEX_GLOBAL 插删后问最小缺失非负数	5
GAME_PN 先枚举胜败状态，再证明规律	5
GAME_NIM 独立子游戏为何异或，Nim 怎样走到败态	5
INT_PLUS 从 0 只做区间+1：正差分是必须新开的层	6
INT_SIGNED 区间±1：两端补 0 后的总变差除以 2	6
INT_LIMITS 只能改前缀或后缀：每个长度的系数被唯一确定	6
CONSTRUCT_FRAME 先从 n=1、2、3 拆限制，再构造并独立验合法	7
SEARCH_CHECK 把互相牵制的优化目标固定下来，再证明单调	7
RT_ANGLE 从向量求距离、夹角与方向	7
RT_SEARCH 实数答案的分界，或函数的唯一谷底	8
RT_DISPLAY 显示几位小数，与数值精度和取整分开考虑	8
SEG-00 一页速查：先选容器，再选维护信息	9
STL_SELECT 先列操作，再选顺序容器、有序集合或堆	9
XOR03 统计 XOR 为 k 的子数组	9
XOR05 贡献：XOR / OR / AND 三种表	10
IO01 原版 ll/getchar 快读	11
FP01 浮点误差、距离与反三角	12
TRS01 实数二分与三分	13
STL01 可执行 STL 容器与算法短例	14
MEX01 / MEX02 静态与前缀 MEX	18
GAME01 正常规则、Nim 胜着与有限 DAG SG	19
BM01 区间单位操作与总变差	20
PG01 树的存储、多测与根化	22
PSD01 Fenwick：个人 add/query/rangeSum	24
X01 前缀 XOR、子数组计数与离线区间异或	25

【MEX_GLOBAL】插删后问最小缺失非负数

识别：mex、最小没有出现的整数；单点修改后问全数组 mex。

怎么推出：静态 vis 扫描；仅插入的前缀维护只增指针；任意插删维护 cnt 与缺失 set。

关键证明：[0,N]有 N+1 个数，N 个元素不能全覆盖；缺失 set 与 cnt==0 互补保持。

别套错：{0,0}删一个 0 后 mex 仍 1；负数不占 mex；所有区间共享一个全局 set 无法回答每个区间的 mex。

练习：cf/3/1096/D_mex_set.cpp::cal 局部模型：中心扩展时把遇到的值从候选 set 中移除；完整题面与线上 AC 未核验。

代码入口：MEX01、MEX02

调用提醒：mex_init(n); for(int x:a) mex_add(x); mex_erase(old); mex_add(value); cout<<mex_query();

原代码：template/mex.cpp；cf/3/1096/D_mex_set.cpp

【GAME_PN】先枚举胜败状态，再证明规律

识别：两人轮流、询问先手输赢/合法胜着。

怎么推出：正常规则终态败；存在败态后继则胜，否则败。小范围表格用于猜奇偶/模数规律，再对全部状态归纳证明。

关键证明：能走到败态就把无解留给对手；所有后继都赢时每一步都给对手赢态。

别套错：取 1 或 2 时败态是 3 的倍数，奇偶猜法在 2 就错；只对前 20 项成立不是证明。

练习：tests/round02/test_game_string.cpp：取{1,3,4}枚举到 50，并验证双游戏的直接 P/N 递推。

代码入口：GAME01

原代码：template/mex.cpp

【GAME_NIM】独立子游戏为何异或，Nim 怎样走到败态

识别：若干堆，每步仅改变一堆；或已证明独立的公平正常子游戏之和。

怎么推出：Nim 堆的 SG 是堆大小；一般有限公平 DAG 取后继 SG 集合的 mex。异或非零找最高为 1 的位，把相应 Nim 堆 ai 减到 ai xor sum。

关键证明：零异或任意合法改变一个 SG 都会非零；非零异或存在一个分量可改变为使总异或零的更小 SG 值。

别套错：一步可同时取两堆时不是独立和；反常规则不能直接套 SG。反常 Nim 只有全 1 时按个数奇偶，其余按 xor，不能推广到任意游戏。

练习：tests/round02/test_game_string.cpp：{1,2,3}败；{1,2,4}将 4 改 3；3 堆 0..5 正常/反常穷举。

代码入口：GAME01、MEX01

原代码：template/mex.cpp

【INT_PLUS】从 0 只做区间+1：正差分是必须新开的层

识别：从全零数组构造非负整数目标，每次给任意连续非空区间加 1，最少多少次；只减 1 的对称版本。

怎么推出：最少次数=a[1]+Σ(i=2..n)max(0,a[i]-a[i-1])。扫描时遇正差分开相应层，负差分关闭相同数量的活跃层；可将相同区间的操作压成次数。只减 1 要求 a[i]<=0，对-a 应用同式。

关键证明：位置 i 比 i-1 多出的高度只能由从 i 开始的新操作提供，正差分和为下界；逐层开闭恰好达到该下界。

别套错：[2,1,3]需 4 次而非 max=3；下降只关层，不另开操作。若每次可加任意整数且统一费用 1，[2,2]只需 1 次，此式失效。取 LLONG_MIN 的负数或差分先提升 i128。

练习：一般练习 LeetCode 1526（官方题面已核验，并非声称用户做过）：[1,2,3,2,1]答案 3；自测[2,1,3]可用[1,3]+1、[1,1]+1、[3,3]+2 达到 4 个单位操作。

代码入口：BM01、BM02

调用提醒：unit_intervals(a,1)计数；construct_unit_intervals(a,1)返回{l,r,sign,cnt}压缩清单。只减 1 传-1。a 含 0 号占位，nullopt 表示符号不可达，合法全零则答案 0、清单空。

原代码：template/competition/models/interval_count.hpp；template/competition/models/interval_construct.hpp

【INT_SIGNED】区间±1：两端补 0 后的总变差除以 2

识别：从 0 变成可正可负的整数数组，每次任选区间加 1 或减 1，求最少单位操作数。

怎么推出：答案=Σmax(0,d[i])=Σ|d[i]|/2=(|a[1]|+Σ(i=2..n)|a[i]-a[i-1]|+|a[n]|)/2。把正差分和负差分逐一配对：正点在左用+区间，在右用-区间。

关键证明：t 次操作形成的差分 L1 范数至多 2t，给出下界；正负边界配对每次恰消掉等量需求，达到下界。

别套错：[1,-1]答案 2，不是只看内部变化得到 1；任意区间改成固定长度、只能前缀、或中间值受限时须重做可达性证明。不能先在 ll 里做 a[i]-a[i-1]再转 i128。

练习：tests/round02/test_models.cpp 把 n<=4、每项[-2,2]全部状态与 BFS 对照，并重放压缩构造；[-2,3]的答案为 5。

代码入口：BM01、BM02

调用提醒：unit_intervals(a,0)与 construct_unit_intervals(a,0)；输出 i128 可用 CO02 内的全局函数 to_string_i128。

原代码：template/competition/models/interval_count.hpp；template/competition/models/interval_construct.hpp；template/tests/round02/test_models.cpp

【INT_LIMITS】只能改前缀或后缀：每个长度的系数被唯一确定

识别：每次只能对[1,r]或[l,n]加 1/减 1；看似区间操作，但一端固定。

怎么推出：前缀±1：|a[n]|+Σ(i=1..n-1)|a[i]-a[i+1]|；后缀±1：|a[1]|+Σ(i=2..n)|a[i]-a[i-1]|。只能+1 时，前缀要求 a[1]>=…>=a[n]>=0 且答案 a[1]；后缀要求 0<=a[1]<=…<=a[n]且答案 a[n]。只-1 对-a 作同一判定。

关键证明：相邻位置相减会消去所有共同覆盖的操作，只留下唯一长度的净系数；最少使用次数至少为系数绝对值，按其符号操作即达到。

别套错：[2,1,3]任意区间+1 需 4 次，前缀±1 需 6 次，后缀±1 需 5 次；不能混用。非负目标[1,2]也不能只靠前缀+1 得到。

练习：tests/round02/test_models.cpp 的 prefix/suffix 独立 BFS；手算[1,2]：前缀±1 为 3，后缀±1 为 2。

代码入口：BM01

调用提醒：prefix_units(a)、suffix_units(a)实现双向计数；仅+1/-1 的额外单调判定属于本卡推导，BM01 没有单向前后缀接口。

原代码：template/competition/models/interval_count.hpp；template/tests/round02/test_models.cpp

【CONSTRUCT_FRAME】先从 n=1、2、3 拆限制，再构造并独立验合法

识别：输出任意合法排列、字符串、图或操作序列；可能要求不存在时输出-1。

怎么推出：优先尝试奇偶分类→镜像/两端向中间→分组配对→按值排序→模类分组→循环移位。每一步写保持的不变量；如要求最少操作，先给下界，再检查构造次数是否等于下界。

关键证明：构造后的每条限制逐条验；对称保证局部相等不等于数量正确。最优构造另需“下界+达到”，不能从一个合法输出推断最优。

别套错：n=1 没有“两端配对”；奇数长度中心只能用一次；移位 k=n 等于没动；排序输出后可能丢失原下标。只验样例或只看形状不能证明合法。

练习：cf/construction/1.cpp 先传播镜像字符再扣计数、处理中心和成对空位；cf/date/5/5.31/3.cpp 原注释明确“先完全再试着补”，用来练分块贡献。原来源线上 AC 均 UNKNOWN。

代码入口：BM02、BM07、STL01、PAL01

调用提醒：BM02 提供可重放的最优单位操作清单；PAL01 保留原 01?镜像填法；STL01 有排列枚举/rotate 短例。独立 checker 检查长度、范围、唯一性、原固定位、目标统计和操作成本。

原代码：cf/construction/1.cpp；cf/date/5/5.31/3.cpp；template/competition/models/interval_construct.hpp

【SEARCH_CHECK】把互相牵制的优化目标固定下来，再证明单调

识别：最小可行值、最大可行值、最小化最大值、最大化最小值；直接优化时变量互相制约。

怎么推出：CF913D 删除不计分题不减分，故可设选 k 题且都计分。check(k)等价于合格题 a[i]>=k 中最小 k 个耗时和<=T；k 可行时删题得到任意更小 k 可行。一般“最大值不超 X”随 X 增大放松，“最小间距至少 X”随 X 增大收紧。

关键证明：从任一可行方案显式构造阈值放松后的方案，证明可行集合是前缀或后缀；二分只丢弃由该单调性排除的一半。

别套错：“恰好等于 x 能做到”不一定单调，例如可达和{0,2,4}；此时不能直接二分。只证明数学可行性单调，尚未证明写出的贪心 check 等价。

练习：用户 cf/other/Hello_2018/D.cpp 及官方 CF913D；重新核对官方第二样例没有题可做时原程序输出 0，且不会误调用 check(0)。

代码入口：BM05、CO03

调用提醒：原 BM05 从 l=1,r=n、ans=0 搜索，check(0)从不调用，原逻辑正确且无需重写。若复用 CO03 泛型接口，必须让传入谓词支持接口可能查询的所有端点。

原代码：cf/other/Hello_2018/D.cpp；template/core/search_window.hpp

【RT_ANGLE】从向量求距离、夹角与方向

识别：两点距离、两个非零向量夹角、点绕原点的方位。

怎么推出：距离用 hypot；方向用 atan2(y,x)后归一化；夹角用单位向量点积再 acos。

关键证明：点积=|a||b|cosθ；atan2 结合 x/y 符号保留象限。

别套错：把 atan2 参数反过来；对零向量强行定义角；把 EPS 加入排序比较器破坏严格弱序。

练习：tests/round02/test_runtime.cpp：三四五距离、四象限、微小越界、2000 随机单位向量。

代码入口：FP01

调用提醒：distance_2d(0,0,3,4)==5；degrees(angle_between(1,0,0,1))≈90；direction_angle(0,-1)≈3π/2。

原代码：template/Ternary_Search.cpp

【RT_SEARCH】实数答案的分界，或函数的唯一谷底

识别：求最小可行实数；或一个连续函数先下降后上升求最小值。

怎么推出：二分比较可行性留下转折点；三分比较 m1/m2 函数值留下最小区间。

关键证明：m1<m2 且 f(m1)<=f(m2)时，单谷性质说明 m2 右侧没有更优最小点。

别套错：f 有两个谷时会删掉全局最优；整数答案不应用浮点二分再直接强转；无限精度迭代并不存在。

练习：template/Ternary_Search.cpp 只借用 200 轮循环，题目几何性质未在本卡确认；测试对 $x^2>=v$ 与已知顶点二次函数。

代码入口：TRS01

原代码：template/Ternary_Search.cpp

【RT_DISPLAY】显示几位小数，与数值精度和取整分开考虑

识别：要求保留 k 位小数，或需要对负数取整；题目涉及距离、比例或角度。

怎么推出：fixed+setprecision(k)固定小数位；单独 setprecision(k)默认有效数字。double 通常约 15 位有效数字，long double 精度由实现决定。floor 向负无穷取整，ceil 向正无穷，trunc 向 0，round 的半数远离 0。

关键证明：格式化在已有二进制浮点值上产生十进制文本，不改变该值；例如-1.5 在四种取整下分别得到-2/-1/-2/-1。

别套错：显示 k 位不是十进制精确舍入的通用解法；float 精度较少；超出整数范围时强转不安全；负数不能把/或强转误当 floor。

练习：tests/round02/test_runtime.cpp 实际执行格式、snprintf 与四种取整断言；本卡 main 另独立编译并核对完整输出。

代码入口：IO02、FP01

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    // 本例保留默认同步；因此 cout/printf 输出顺序可一起演示。
    double x = 12.3456;
    cout << setprecision(3) << x << '\n';
    cout << fixed << setprecision(3) << x << '\n';
    printf("%.10f\n", 0.125);
    printf("%.3Lf\n", 1.0L / 8);
    long double y = -1.5L;
    cout << setprecision(0) << floor(y) << ' ' << ceil(y) << ' '
         << round(y) << ' ' << trunc(y) << '\n';
}
```

原代码：template/core/main.cpp；template/Ternary_Search.cpp

【SEG-00】一页速查：先选容器，再选维护信息

识别：看到区间修改或区间询问时，先判断是否在线、操作能否合并。

怎么推出：静态区间和→前缀和；只在最后要结果的区间加→差分。；单点加 + 前缀/区间和→Fenwick：前缀可相减；sum 以外的操作先检查逆运算。；静态 min/max/gcd→ST 表：可合并且重复覆盖不改变结果；静态 sum 不能用重叠两块相加。；固定长度窗口 min/max→单调队列：每个下标至多进出一次；不是任意区间结构。；动态任意区间、复杂 Node→线段树；整段更新还需能 O(1)作用于 Node 的 lazy。；点改 sum/min/max/gcd→一个字段；max+次数→mx,cnt；最大子段→sum,pref,suff,best。；区间加→sum 加 len*x、min/max 各加 x；区间赋值→has_set，不能用 0 代表无标记。；set 后 add 和 add 后 set 不同：下传先 set 再 add；kth 必须非负频率；阈值搜索明确[L,R]。

关键证明：先利用操作的额外限制减少维护量；没有这些限制时用可合并区间摘要，且以结合律保证划分无关。

别套错：前缀 min 相减无意义：前缀 min 相同也不能推出中间最小值。；ST 两块若求和会重复计数重叠部分。；普通 gcd/最大子段 Node 不能直接接上区间加 lazy。

练习：对同一数组分别加入：无修改、点加、区间加、固定窗口四种约束，先写出能淘汰哪些结构。

代码入口：PSD01、PSD02、PSD10、PSD11、DS04、DS05

原代码：template/Fenwick_tree.cpp；template/segmen tree1.cpp；template/Segment tree2.cpp；cf/date/6/6.9/1_special_segment_tree.cpp

【STL_SELECT】先列操作，再选顺序容器、有序集合或堆

识别：题目反复加入、删除、取最值、计频或查阈值。

怎么推出：顺序表 vector；两端 deque；最近未匹配 stack；按到达处理 queue；只取最优 priority_queue；有序阈值 set/multiset；键到值 map；仅等值查询 unordered。

关键证明：先确认每次操作需要的信息，所选结构必须支持全部必要操作，不因某一个 O(1)接口忽略其余昂贵操作。

别套错：堆不支持有序前驱；set 不提供 O(1)第 k 项；unordered 没有排序；vector 中间 erase 搬移 O(n)。

练习：template/data_sturcture.cpp 原示例扩展为 STL01 可执行场景。

代码入口：STL01、STL02

调用提醒：STL01 所有短例已实际运行；STL02 给出对应模型函数。

原代码：template/data_sturcture.cpp

【XOR03】统计 XOR 为 k 的子数组

识别：问多少个连续区间 XOR 等于 0/k，n 达到 2e5。

怎么推出：cnt[0]=1；逐个计算 p，ans+=cnt[p^k]，再 cnt[p]++。

关键证明：每个右端恰把所有满足等式的历史前缀配对一次，不遗漏也不重计。

别套错：先插后查会在 k=0 多算 n 个空区间。；n 个 0 且 k=0 答案 n(n+1)/2，不是 n。

练习：局部练习：a=[0,0,0],k=0 得 6；a=[3,1,2],k=0 得 1，不冒充 2268C 原题。

代码入口：X01

原代码：cf/1/1124/C.cpp

【XOR05】贡献：XOR / OR / AND 三种表

识别：所有数对求位运算值的普通和；或所有子数组的 XOR 聚合。

怎么推出：无序 i<j：XOR=one*zero；AND=C(one,2)；OR=one*zero+C(one,2)，再乘 2^k。；子数组 XOR 普通和：把 n+1 个前缀套同一位的不同数对计数。；所有子数组 XOR 再 XOR：a[i]出现 i(n-i+1)次，仅两因子都奇时保留。

关键证明：每个数对的每位独立，加法求和可以交换求和顺序。每对不同前缀按较早/较晚唯一对应一个非空区间。

别套错：普通加法有进位，但贡献按权重求和仍正确；最大值目标则不能逐位独立选不同数对。；{1,2,3}的数对 XOR/OR/AND 和分别 6/9/3。

练习：从 cf/4/1090/E 的 n^2枚举迁移；本卡是求和扩展，不将原 max 题误写成 sum。

代码入口：X02

原代码：cf/4/1090/E_xor_find_pattern.cpp

【IO01】原版 ll/getchar 快读

识别：读入格式已知、整数个数已知、需要快速读普通合法整数。

证明：每读入一位 d 后 x=10*x+d，循环结束时 x 等于数字绝对值；最后乘符号。

调用：ll n=read(); for(int i=1;i<=n;++i) a[i]=read();

复杂度：每个字符 O(1)，额外空间 O(1)。

来源：template/read.cpp

规范源码：read.cpp；完整独立代码如下；测试：tests/round02/test_runtime.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;


inline ll read() {
    ll x = 0, f = 1; char ch = getchar();
    while(ch < '0' || ch > '9') {if(ch == '-') f = -1; ch = getchar();}
    while(ch >= '0' && ch <= '9') {x = (x << 3) + (x << 1) + (ch ^ 48); ch = getchar();}
    return x * f;
}
```

【FP01】浮点误差、距离与反三角

识别：二维距离、向量夹角、方向角和数值比较。

证明：点积除以模长积是夹角余弦；atan2 同时利用 x/y 的符号确定象限。

约定：long double，输入与中间结果有限；几何向量非零且 hypot 结果有限；acos/asin 数学参数本来应在[-1,1]。EPS 默认 1e-12，需按误差预算调整。

调用：auto d=distance_2d(0,0,3,4); auto rad=angle_between(1,0,0,1); auto deg=degrees(rad); auto dir=direction_angle(0,-1);

复杂度：各函数 O(1)。

来源：template/Ternary_Search.cpp

规范源码：competition/runtime/floating.hpp；完整独立代码如下；测试：tests/round02/test_runtime.cpp

```
#include <bits/stdc++.h>
using namespace std;

// EPS 随量级/误差预算选；排序比较器仍用精确 <，不要掺 EPS。
constexpr long double EPS = 1e-12L;
const long double PI = acosl(-1.0L);
inline bool fp_equal(long double a, long double b) {
    return fabsl(a - b) <= EPS * max({1.0L, fabsl(a), fabsl(b)});
}
inline long double radians(long double deg) { return deg * PI / 180; }
inline long double degrees(long double rad) { return rad * 180 / PI; }
inline long double normalize_angle(long double rad) {
    rad = fmodl(rad, 2 * PI);
    if (rad < 0) rad += 2 * PI;
    if (rad >= 2 * PI) rad = 0; // 极小负数加一周可能舍入成 2π。
    return rad;
}
// 只修正舍入造成的微小越界；原数学输入本来就必须在 [-1,1]。
inline long double safe_acos(long double x) {
    return acosl(clamp(x, -1.0L, 1.0L));
}
inline long double safe_asin(long double x) {
    return asinl(clamp(x, -1.0L, 1.0L));
}
inline long double distance_2d(long double x1, long double y1,
                               long double x2, long double y2) {
    return hypotl(x1 - x2, y1 - y2);
}
// (x,y) 非零；atan2 的参数是 (y,x)，返回方向角 [0,2π)。
inline long double direction_angle(long double x, long double y) {
    assert(x != 0 || y != 0);
    return normalize_angle(atan2l(y, x));
}
// 两个向量均非零且有限；先单位化，避免直接点积/模长乘积溢出。
inline long double angle_between(long double ax, long double ay,
                                 long double bx, long double by) {
    long double na = hypotl(ax, ay), nb = hypotl(bx, by);
    assert(na > 0 && nb > 0 && isfinite(na) && isfinite(nb));
    return safe_acos((ax / na) * (bx / nb) + (ay / na) * (by / nb));
}
```

【TRS01】实数二分与三分

识别：实数答案可行性单调，或目标函数在区间内单谷。

证明：若 m1<m2 且 f(m1)<=f(m2)，单谷且无非最优平台保证右侧 m2 以外不会更优，因此可令 r=m2。

约定：二分 check 仅 false->true 且右端可行；三分函数连续，最小区间外严格下降/上升，可全程单调，不允许非最优平台。边界和 r-l 有限；check/f 数值可靠。

调用：auto x=real_first_true(0,2,[](long double t){return t*t>=2;}); auto p=ternary_min(-100,100,[](long double t){return (t-3)*(t-3)+7;});

复杂度：固定 200 轮，O(200*一次 check/f)，空间 O(1)。

来源：template/Ternary_Search.cpp

规范源码：competition/runtime/real_search.hpp；完整独立代码如下；测试：tests/round02/test_runtime.cpp

```
#include <bits/stdc++.h>
using namespace std;

// check 在 [l,r] 上只能 false -> true，check(r) 为 true。
// 返回转折点近似值；边界、r-l、check 运算须有限。
template<class Check>
long double real_first_true(long double l, long double r, Check check) {
    assert(l <= r && check(r));
    for (int it = 0; it < 200; ++it) {
        long double mid = l + (r - l) / 2;
        if (check(mid)) r = mid;
        else l = mid;
    }
    return r;
}
// f 连续且先严格下降后严格上升，可有连续平底最小区间。
// 也允许全程单调；不得在非最优值处存在平坦区间。
// 数值噪声/多峰函数无保证；200 次不能修复错误模型。
template<class Function>
long double ternary_min(long double l, long double r, Function f) {
    assert(l <= r);
    for (int it = 0; it < 200; ++it) {
        long double m1 = l + (r - l) / 3;
        long double m2 = r - (r - l) / 3;
        if (f(m1) <= f(m2)) r = m2;
        else l = m1;
    }
    return l + (r - l) / 2;
}
```

【STL01】可执行 STL 容器与算法短例

识别：选择容器、排序/二分/去重/计数、读写字符串，现场查小段调用。

证明：比较器保持严格弱序才能排序；排序后相同元素相邻，unique+erase 才能得到去重序列；lower/upper 界定整个等值段。

约定：GNU++20；每个花括号是独立短例，断言给出预期。算法区间半开；查询前满足排序/非空/下标等条件。示例不是统一封装类。

调用：stl_container_recipes(); stl_algorithm_recipes(); stl_string_recipes(); // 学习/验证；比赛抄所需局部花括号。

复杂度：array/vector 访问 O(1)；有序集合 O(log n)；哈希平均 O(1)最坏 O(n)；sort O(n log n)；算法扫描 O(n)，nth_element 平均 O(n)。

来源：template/data_sturcture.cpp

规范源码：competition/stl/recipes.hpp；完整独立代码如下；测试：tests/round02/test_stl.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 可执行短例：每个花括号可单独抄用；assert 给出具体结果。
inline void stl_container_recipes() {
    { // array / vector：定长与动态序列。
        array<int, 4> a{3, 1, 2, 0};
        sort(a.begin(), a.end());
        assert(a.front() == 0 && a.back() == 3);
        a.fill(7); assert(a[2] == 7);
        vector<int> b; b.reserve(8); // reserve 不改变 size。
        b.push_back(3); b.emplace_back(4); b.resize(4, 9);
        b.pop_back(); b.erase(b.begin());
        assert((b == vector<int>{4, 9}));
        b.clear(); assert(b.empty()); // 多测清逻辑长度，容量可复用。
    }
    { // deque：两端增删，仍可 O(1)按下标访问。
        deque<int> q;
        q.push_back(2); q.push_front(1); q.push_back(3);
        q.pop_front(); q.pop_back();
        assert(q.size() == 1 && q[0] == 2);
    }
    { // pair / tuple：字典序排序和结构化绑定。
        vector<pair<int, int>> tasks{{2, 1}, {1, 7}, {1, 3}};
        sort(tasks.begin(), tasks.end());
        auto [time, id] = tasks.front();
        auto state = make_tuple(time, id, string("ready"));
        auto [t, i, label] = state;
        assert(t == 1 && i == 3 && label == "ready");
        assert(get<1>(state) == 3);
    }
    { // stack：左括号等待最近的右括号。
        string s = "(()())";
        stack<int> left;
        bool ok = true;
        for (int i = 0; i < int(s.size()); ++i) {
            if (s[i] == '(') left.push(i);
            else if (left.empty()) { ok = false; break; }
            else left.pop();
        }
        assert(ok && left.empty());
    }
    { // queue：先进先出，BFS 已发现结点的处理顺序。
        queue<int> q;
        q.push(1); q.push(2);
        vector<int> order;
        while (!q.empty()) { order.push_back(q.front()); q.pop(); }
        assert((order == vector<int>{1, 2}));
    }
    { // 三种常用堆，以及结构体自定义优先级。
```

【STL01】可执行 STL 容器与算法短例 续

```
        priority_queue<ll> big;
        priority_queue<ll, vector<ll>, greater<ll>> small;
        priority_queue<pair<ll, int>, vector<pair<ll, int>>,
                       greater<pair<ll, int>>> pairs;
        for (ll x : {2LL, 8000000000LL, 3LL}) { big.push(x); small.push(x); }
        assert(big.top() == 8000000000LL && small.top() == 2);
        pairs.push({4000000000LL, 9}); pairs.push({4000000000LL, 3});
        assert(pairs.top() == make_pair(4000000000LL, 3));
        struct Job { int deadline, id; };
        auto cmp = [](const Job& a, const Job& b) {
            return tie(a.deadline, a.id) > tie(b.deadline, b.id);
        };
        priority_queue<Job, vector<Job>, decltype(cmp)> pq(cmp);
        pq.push({4, 1}); pq.push({2, 9}); pq.push({2, 3});
        assert(pq.top().deadline == 2 && pq.top().id == 3);
    }
    { // set 去重；multiset 保留重复；删除一个要用迭代器。
        set<int> s{3, 1, 3}; assert(s.size() == 2);
        assert(s.contains(1) && !s.contains(2)); // C++20 等值存在性。
        assert(s.find(1) != s.end() && s.find(9) == s.end());
        s.insert(2); assert(s.count(2) == 1);
        assert(s.erase(2) == 1); // set 无重复；按值删除返回删除数。
        auto it = s.lower_bound(2); assert(it != s.end() && *it == 3);
        multiset<int> ms{2, 2, 5};
        auto pos = ms.find(2);
        if (pos != ms.end()) ms.erase(pos);
        assert(ms.count(2) == 1); // ms.erase(2)会删所有 2。
        ms.insert(2);
        assert(ms.erase(2) == 2 && !ms.contains(2)); // 按值删全部副本。
    }
    { // map 计频；multimap 存同一时间的多项事件。
        map<string, int> cnt;
        for (string x : {"b", "a", "b"}) ++cnt[x];
        assert(cnt.begin()->first == "a" && cnt.at("b") == 2);
        assert(cnt.find("c") == cnt.end()); // 查询不想插入时不用[]。
        multimap<int, string> events{{2, "B"}, {1, "A"}, {2, "C"}};
        auto [l, r] = events.equal_range(2);
        int total = 0;
        for (auto it = l; it != r; ++it) ++total;
        assert(total == 2);
    }
    { // 哈希去重/计频；只要等值查询，不要求前驱或排序。
        unordered_set<int> seen;
        unordered_map<int, int> cnt;
        seen.max_load_factor(0.7f); seen.reserve(100);
        cnt.max_load_factor(0.7f); cnt.reserve(100);
        for (int x : {4, 4, -1}) { seen.insert(x); ++cnt[x]; }
        assert(seen.size() == 2 && cnt.at(4) == 2);
    }
}

inline void stl_algorithm_recipes() {
    { // 自定义结构体排序；同分按 id 升序，必须严格小于。
        struct Node { int score, id; };
        vector<Node> a{{9, 2}, {7, 1}, {9, 1}};
        sort(a.begin(), a.end(), [](const Node& x, const Node& y) {
            if (x.score != y.score) return x.score > y.score;
            return x.id < y.id;
        });
        assert(a[0].id == 1 && a[1].id == 2);
        stable_sort(a.begin(), a.end(), [](const Node& x, const Node& y) {
            return x.score < y.score;
        });
        assert(a[0].id == 1 && a[1].id == 1 && a[2].id == 2);
    }
    { // 升序找等值段；降序必须传同一比较器。
        vector<int> a{1, 2, 2, 5};
```

【STL01】可执行 STL 容器与算法短例 续

```
        auto l = lower_bound(a.begin(), a.end(), 2);
        auto r = upper_bound(a.begin(), a.end(), 2);
        assert(r - l == 2 && binary_search(a.begin(), a.end(), 5));
        sort(a.begin(), a.end(), greater<int>());
        auto dl = lower_bound(a.begin(), a.end(), 2, greater<int>());
        auto dr = upper_bound(a.begin(), a.end(), 2, greater<int>());
        assert(dl - a.begin() == 1 && dr - a.begin() == 3);
        assert(binary_search(a.begin(), a.end(), 5, greater<int>()));
    }
    { // 第 k 小只需分隔，不要求其余位置有序；k 为 0-based。
        vector<int> a{7, 1, 4, 4, 9};
        int k = 2;
        nth_element(a.begin(), a.begin() + k, a.end());
        assert(a[k] == 4);
        for (int i = 0; i < k; ++i) assert(a[i] <= a[k]);
        for (int i = k + 1; i < int(a.size()); ++i) assert(a[i] >= a[k]);
    }
    { // unique 只压相邻重复；erase 才实际缩短容器。
        vector<int> a{3, 1, 3, 2, 2};
        sort(a.begin(), a.end());
        a.erase(unique(a.begin(), a.end()), a.end());
        assert((a == vector<int>{1, 2, 3}));
        a.erase(remove_if(a.begin(), a.end(),
            [](int x) { return x % 2; }), a.end());
        assert((a == vector<int>{2}));
    }
    { // 反转/循环左移；排列枚举必须包含初始排列。
        vector<int> a{1, 2, 3, 4};
        reverse(a.begin(), a.end());
        rotate(a.begin(), a.begin() + 1, a.end());
        assert((a == vector<int>{3, 2, 1, 4}));
        string p = "aab";
        int ways = 0;
        do { ++ways; } while (next_permutation(p.begin(), p.end()));
        assert(ways == 3); // 初始有序时，重复元素不会重复枚举。
        assert(prev_permutation(p.begin(), p.end()) == false);
    }
    { // 64 位前缀/后缀；partial_sum 的内部类型取决于输入类型。
        vector<long long> a{2000000000LL, 2000000000LL, -1};
        auto sum = accumulate(a.begin(), a.end(), 0LL);
        vector<long long> pre(a.size()), suf(a.size());
        partial_sum(a.begin(), a.end(), pre.begin());
        partial_sum(a.rbegin(), a.rend(), suf.rbegin());
        assert(sum == 3999999999LL && pre[1] == 4000000000LL);
        assert(suf[1] == 1999999999LL);
        assert(*min_element(a.begin(), a.end()) == -1);
        assert(*max_element(a.begin(), a.end()) == 2000000000LL);
    }
    { // 递归 lambda：参数 self 传自身，枚举每项选/不选。
        vector<int> a{1, 3}, sums;
        auto dfs = [&](auto&& self, int i, int sum) -> void {
            if (i == int(a.size())) { sums.push_back(sum); return; }
            self(self, i + 1, sum);
            self(self, i + 1, sum + a[i]);
        };
        dfs(dfs, 0, 0); sort(sums.begin(), sums.end());
        assert((sums == vector<int>{0, 1, 3, 4}));
    }
}
```

【STL01】可执行 STL 容器与算法短例 续

```
inline void stl_string_recipes() {
    string s = "abC abC";
    size_t pos = s.find("abC", 1);
    assert(pos != string::npos && pos == 4);
    string part = s.substr(pos, 3); // 第二参数是长度，不是右端点。
    reverse(part.begin(), part.end());
    assert(part == "Cba");
    for (char& c : part) c = char(tolower((unsigned char)c));
    assert(part == "cba");
    for (char& c : part) c = char(toupper((unsigned char)c));
    assert(part == "CBA"); // 按字节/当前 locale；不处理 UTF-8 汉字。
    assert(s.find("xyz") == string::npos);
}
```

【MEX01 / MEX02】静态与前缀 MEX

识别：问最小未出现的非负整数；只有插入的前缀序列。

证明：鸽巢原理给 mex<=n；vis 精确记录相关值，最小未标记位置即定义。

约定：vector<int>可含负数和很大值；静态 n 个数仅需看[0,n]。前缀不删除。

调用：int x=static_mex(a); auto pm=prefix_mex(a);

复杂度：静态 O(n)时间空间；所有前缀合计 O(n)，因为 now 只向右移动。

来源：template/mex.cpp；cf/3/1096/D_mex_set.cpp

规范源码：competition/game/mex.hpp；完整独立代码如下；测试：tests/round02/test_game_string.cpp

```
#include <bits/stdc++.h>
using namespace std;

// n 个元素的 mex 在 [0,n]；负数和大于 n 的值不影响答案。
inline int static_mex(const vector<int>& a) {
    int n = int(a.size());
    vector<bool> vis(n + 1);
    for (int x : a) if (0 <= x && x <= n) vis[x] = true;
    for (int x = 0; x <= n; ++x) if (!vis[x]) return x;
    return n + 1; // n 个元素不可能占满 n+1 个不同值。
}
inline vector<int> prefix_mex(const vector<int>& a) {
    int n = int(a.size()), now = 0;
    vector<bool> vis(n + 1);
    vector<int> ans;
    for (int x : a) {
        if (0 <= x && x <= n) vis[x] = true;
        while (now <= n && vis[now]) ++now;
        ans.push_back(now);
    }
    return ans;
}

// 动态多重集合：全过程元素总数不超过 mex_limit。
// init(N) 建 [0,N] 缺失值；忽略区间外值；删值必须确实存在。
inline int mex_limit;
inline vector<int> cnt;
inline set<int> missing;
inline void mex_init(int limit) {
    assert(limit >= 0);
    mex_limit = limit;
    cnt.assign(limit + 1, 0);
    missing.clear();
    for (int x = 0; x <= limit; ++x) missing.insert(x);
}
inline void mex_add(int x) {
    if (x < 0 || x > mex_limit) return;
    if (cnt[x]++ == 0) missing.erase(x);
}
inline void mex_erase(int x) {
    if (x < 0 || x > mex_limit) return;
    assert(cnt[x] > 0);
    if (--cnt[x] == 0) missing.insert(x);
}
inline int mex_query() {
    assert(!missing.empty());
    return *missing.begin();
}
```

【GAME01】正常规则、Nim 胜着与有限 DAG SG

识别：两人轮流、完全信息、可枚举合法后继；独立取石堆或可拆成独立公平子游戏。

证明：Nim 异或非零时取最高为 1 的位所在堆，将 ai 改 ai xor sum 会严格变小并使总异或为 0；任何一步都会破坏原来的零异或。

约定：正常规则无棋可走者输；SG 图有限无环，状态编号保证后继 u<v。异或定理要求每步仅改变一个独立子游戏，双方可走集合相同，无随机。反常 Nim 函数仅用于取任意正石的 Nim 规则。

调用：auto [i,left]=nim_winning_move(a); if(i!=-1) a[i]=left; auto sg=sg_ordered_dag(moves); bool win=(sg[s1]^sg[s2])!=0;

复杂度：Nim O(堆数)；SG O(状态数+边数)，额外 O(状态数+最大出度)。

来源：template/mex.cpp

规范源码：competition/game/normal_play.hpp；完整独立代码如下；测试：tests/round02/test_game_string.cpp

```
#include <bits/stdc++.h>
using namespace std;

// 正常 Nim：每步选择一堆，取走任意正数；无棋可走者输。
inline unsigned long long nim_xor(const vector<unsigned long long>& a) {
    unsigned long long sum = 0;
    for (auto x : a) sum ^= x;
    return sum;
}
// 返回 {0-based 堆下标，操作后堆大小}；必败态返回 {-1,0}。
inline pair<int, unsigned long long>
nim_winning_move(const vector<unsigned long long>& a) {
    auto sum = nim_xor(a);
    if (!sum) return {-1, 0};
    for (int i = 0; i < int(a.size()); ++i)
        if ((a[i] ^ sum) < a[i]) return {i, a[i] ^ sum};
    return {-1, 0};
}

// 有限、无环、公平、无随机、完全信息、正常规则的有向状态图。
// 状态已按依赖编号：moves[v] 内每个 u 必须满足 0<=u<v。
// sg[v]=mex({sg[u]})；独立子游戏之和的 SG 才可异或。
inline vector<int> sg_ordered_dag(const vector<vector<int>>& moves) {
    int n = int(moves.size());
    vector<int> sg(n);
    for (int v = 0; v < n; ++v) {
        vector<bool> vis(moves[v].size() + 1);
        for (int u : moves[v]) {
            assert(0 <= u && u < v);
            if (sg[u] < int(vis.size())) vis[sg[u]] = true;
        }
        while (sg[v] < int(vis.size()) && vis[sg[v]]) ++sg[v];
    }
    return sg;
}

// 反常 Nim：拿最后一个的人输，仅限 Nim 取石规则。
// 若所有非空堆都是 1，偶数个 1 必胜；否则仍看 xor。
// 全空时返回 true：按反常约定，无棋可走者赢。
inline bool misere_nim_win(const vector<unsigned long long>& a) {
    int ones = 0;
    bool large = false;
    for (auto x : a) {
        ones += (x == 1);
        large |= (x > 1);
    }
    return large ? nim_xor(a) != 0 : ones % 2 == 0;
}
```

【BM01】区间单位操作与总变差

识别：从零变目标，问任意区间/前缀/后缀±1 或二进制翻转的最少次数；相邻转移。

证明：每次单位区间操作至多消去一单位正差分，故正差分和是下界；正负边界配对的区间构造达到它。前后缀系数则由相邻差唯一确定。

约定：a[0]=0、a[1..n]为 ll；任意区间 mode=1 要求非负目标，-1 要求非正，0 允许任意符号且中间状态无额外限制；计数 i128。相邻转移要求总和相同。

调用：auto ans=unit_intervals({0,2,1,3},1); // 4；prefix_units({0,2,1,3})=6；suffix_units(...)=5

复杂度：每个计数函数 O(n)时间、O(1)额外空间。

来源：cf/date/6/6.9/1_special_segment_tree.cpp

规范源码：competition/models/interval_count.hpp；完整独立代码如下；测试：tests/round02/test_models.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using i128 = __int128_t;
// a[1..n], a[0]=0。中间值和操作次数用 i128。
// mode=1 只加 1；-1 只减 1；0 两者均可。无其他中间状态限制。
optional<i128> unit_intervals(const vector<ll>& a, int mode = 0) {
    assert(!a.empty() && -1 <= mode && mode <= 1);
    i128 ans = 0, pre = 0;
    for(int i = 1; i < (int)a.size(); i++) {
        if((mode == 1 && a[i] < 0) || (mode == -1 && a[i] > 0))
            return nullopt;
        i128 d = (i128)a[i] - pre;
        if(d > 0) ans += d;
        pre = a[i];
    }
    if(pre < 0) ans -= pre; // 补上最后边界 d[n+1]=-a[n]
    return ans;
}
// 只能操作前缀/后缀，每次可加 1 或减 1。
i128 prefix_units(const vector<ll>& a) {
    i128 ans = 0, next = 0;
    for(int i = (int)a.size() - 1; i >= 1; i--) {
        i128 d = (i128)a[i] - next;
        ans += d < 0 ? -d : d;
        next = a[i];
    }
    return ans;
}
i128 suffix_units(const vector<ll>& a) {
    i128 ans = 0, pre = 0;
    for(int i = 1; i < (int)a.size(); i++) {
        i128 d = (i128)a[i] - pre;
        ans += d < 0 ? -d : d;
        pre = a[i];
    }
    return ans;
}
// 从全 0 二进制数组出发，任意区间按位翻转：答案是连续 1 段数。
int binary_interval_flips(const vector<int>& a) {
    int ans = 0, pre = 0;
    for(int i = 1; i < (int)a.size(); i++) {
        assert(a[i] == 0 || a[i] == 1);
        if(a[i] == 1 && pre == 0) ans++;
        pre = a[i];
    }
    return ans;
}
// 相邻位置转移 1 单位；允许两个方向。总和相同才可达。
```

【BM01】区间单位操作与总变差 续

```
optional<i128> adjacent_moves(const vector<ll>& a, const vector<ll>& b) {
    assert(a.size() == b.size());
    i128 pre = 0, ans = 0;
    for(int i = 1; i < (int)a.size(); i++) {
        pre += (i128)a[i] - b[i];
        if(i + 1 < (int)a.size()) ans += pre < 0 ? -pre : pre;
    }
    if(pre != 0) return nullopt;
    return ans;
}
```

【PG01】树的存储、多测与根化

识别：n 个点 n-1 条无向边；先建立树，再求父亲、深度、子树大小或树 DP。

证明：连通无向图有 n-1 条边当且仅当是树。入栈时标记保证每点唯一父亲；父先子后的顺序逆转后，所有孩子先于父亲完成累加。

约定：1<=n<200010；g[1..n]是双向对称邻接表，端点合法。read_tree 读 n 及 n-1 条边并先 init。root_tree(root)非树返回 false；成功时根 fa=0、dep=0，sz 包括自己，order 父亲先于儿子。

调用：istringstream in("4 1 2 2 3 2 4");
read_tree(in);
bool ok=root_tree(1);
// ok && sz[2]==3

复杂度：建图/根化 O(n)，空间 O(n)。

来源：template/tree_storage.cpp；cf/3/1096/H_!!dp_tree.cpp

规范源码：competition/graph/tree_storage.hpp；完整独立代码如下；测试：tests/round02/test_graph_personal.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// PG01. 原 tree_storage.cpp 的双向邻接表；点 1..n，1<=n<N。
const int N = 200010;
int n;
vector<ll> g[N];
ll fa[N], dep[N], sz[N];
vector<ll> order; // 父亲先于儿子；逆序可做子树 DP。

void init(int count) {
    n = count;
    for (int i = 1; i <= n; ++i) {
        g[i].clear();
        fa[i] = sz[i] = 0;
        dep[i] = -1;
    }
    order.clear();
}

void read_tree(istream& in = cin) {
    int count;
    in >> count;
    init(count);
    for (int i = 1; i <= n - 1; ++i) {
        ll u, v;
        in >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
}

// 双向对称的邻接表由调用方保证。非树返回 false。
bool root_tree(int root = 1) {
    order.clear();
    if (root < 1 || root > n) return false;
    ll edges = 0;
    for (int i = 1; i <= n; ++i) {
        fa[i] = 0; dep[i] = -1; sz[i] = 1;
        edges += (ll)g[i].size();
    }
    if (edges != 2LL * (n - 1)) return false;
    vector<ll> st{root};
    dep[root] = 0;
    while (!st.empty()) {
        ll u = st.back(); st.pop_back();
        order.push_back(u);
```

【PG01】树的存储、多测与根化 续

```
        for (ll v : g[u]) if (dep[v] == -1) {
            fa[v] = u; dep[v] = dep[u] + 1;
            st.push_back(v);
        }
    }
    if ((int)order.size() != n) return false;
    for (int i = n - 1; i > 0; --i) {
        ll u = order[i];
        sz[fa[u]] += sz[u];
    }
    return true;
}
```

【PSD01】Fenwick：个人 add/query/rangeSum

识别：单点加、前缀和、可相减的区间和。

证明：query 访问的 lowbit 块恰好划分[1,x]，两个前缀相减得到闭区间和。

约定：0<=n<2^30 且内存足够；1 下标，add 只能 1..n；query(0)=0；空 rangeSum 返回 0；所有中间量在 ll 内。

调用：Fenwick fen(n); fen.add(pos,delta); fen.query(r); fen.rangeSum(l,r); 完整 main 见 competition/data_structure/examples/fenwick.cpp

复杂度：更新/查询 O(log n)，逐点初始化 O(n log n)，空间 O(n)。

来源：template/Fenwick_tree.cpp

规范源码：competition/data_structure/fenwick.hpp；完整独立代码如下；测试：tests/round02/test_segment.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

// 保留旧 Fenwick 的 n、tr、add 和 lowbit 循环。
// 1 下标；add 是增量，query(x) 是前缀，rangeSum(l,r) 是闭区间。
// 0<=n<2^30 且内存足够；所有节点、前缀及中间值须在 ll 范围。
struct Fenwick {
    int n;
    vector<ll> tr;
    explicit Fenwick(int size) : n(size), tr(size + 1) {
        assert(size >= 0);
    }
    void add(int x, ll val) {
        assert(1 <= x && x <= n);
        for (; x <= n; x += x & -x) tr[x] += val;
    }
    ll query(int x) const {
        assert(0 <= x && x <= n);
        ll res = 0;
        for (; x; x -= x & -x) res += tr[x];
        return res;
    }
    ll rangeSum(int l, int r) const {
        if (l > r) return 0;
        assert(1 <= l && r <= n);
        return query(r) - query(l - 1);
    }
};
```

接在上述代码后面的完整 main（已独立运行样例）：

样例输入：3 4 / 3 -2 4 / 2 1 3 / 1 2 5 / 3 2 / 2 2 3 /

样例输出：5 / 6 / 7 /

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n >> q;
    Fenwick fen(n);
    for (int i = 1; i <= n; ++i) { ll x; cin >> x; fen.add(i, x); }
    while (q--) {
        int op, l, r; ll val;
        cin >> op;
        if (op == 1) { cin >> l >> val; fen.add(l, val); }
        else if (op == 2) { cin >> l >> r; cout << fen.rangeSum(l, r) << '\n'; }
        else { cin >> r; cout << fen.query(r) << '\n'; }
    }
}
```

【X01】前缀 XOR、子数组计数与离线区间异或

识别：连续区间 XOR 查询、统计 XOR 为 k 的子数组，或全部更新后只输出最终数组。

证明：p[r]^p[l-1]中 1..l-1 出现两次。p[r]^p[l-1]=k 等价于 p[l-1]=p[r]^k，只统计过去前缀。差分标记在 l 开始、r+1 取消。

约定：a 至少为{0}，采用 a[1..n]且 n<=1e9；值为 unsigned long long 完整 64 位。range_xor 仅接收合法非空区间；离线操作{l,r,x}满足 1<=l<=r<=n。{}不是空数组表示。

调用：vector<ull> a={0,3,1,2}; auto p=xor_prefix(a); ull x=range_xor(p,1,3); ll cnt=count_subarray_xor(a,0); a=xor_offline_update(a,{{1,2,7}});

复杂度：前缀 O(n)/查询 O(1)；哈希计数期望 O(n)、最坏 O(n²)；离线更新 O(n+q)。空间 O(n)。

来源：cf/1/1124/C.cpp

规范源码：competition/xor/prefix_xor.hpp；完整独立代码如下；测试：tests/round02/test_xor.cpp

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;

// a[1..n]; a[0] 不使用。来源：cf/1/1124/C.cpp 的前缀抵消观察。
vector<ull> xor_prefix(const vector<ull>& a) {
    vector<ull> p(a.size());
    for(size_t i = 1; i < a.size(); i++) p[i] = p[i - 1] ^ a[i];
    return p;
}
ull range_xor(const vector<ull>& p, int l, int r) {
    return p[r] ^ p[l - 1]; // 1 <= l <= r <= n
}
ll count_subarray_xor(const vector<ull>& a, ull k) {
    unordered_map<ull, ll> cnt;
    cnt[0] = 1;
    ull p = 0;
    ll ans = 0;
    for(size_t i = 1; i < a.size(); i++) {
        p ^= a[i];
        auto it = cnt.find(p ^ k);
        if(it != cnt.end()) ans += it->second;
        cnt[p]++; // 先查询，避免 k=0 时计入空区间
    }
    return ans;
}
// 操作 {l,r,x}：将 a[l..r] 都 ^= x。仅需最终数组时使用。
vector<ull> xor_offline_update(vector<ull> a,
                               const vector<array<ull, 3>>& q) {
    vector<ull> d(a.size() + 1);
    for(auto [l, r, x] : q) {
        d[l] ^= x;
        d[r + 1] ^= x;
    }
    ull tag = 0;
    for(size_t i = 1; i < a.size(); i++) {
        tag ^= d[i];
        a[i] ^= tag;
    }
    return a;
}
```